Assignment 2: Git & GitHub Fundamentals (Group D)

Group Size: 3 members registered, 2 active (followed the 2-member path)

Chosen Theme: Unit Converter

Full Names: Mohamed Amine Bouamri (Member A), Myriad Padri (Member B)

Student IDs: 158425 (Bouamri), 162542 (Padri)

Course Codes & Sections: CSC 3324 section 02 (Bouamri), CSC 3324 section 01 (Padri)

GitHub Usernames: Nietzsche-god (Bouamri), mimou777 (Padri)

GitHub Repository URL: https://github.com/Nietzsche-god/unit-converter

GitHub Pages URL: https://nietzsche-god.github.io/unit-converter/

1. Project & Role Distribution

We built a small Python **Unit Converter** with pytest tests, a GitHub Actions workflow and a GitHub Pages project page. Our group had three registered members, but the third member was absent, so we followed the **2-member path**: two comparable feature areas and at least 3 unit tests per member.

- **Member A (Mohamed Amine Bouamri):** Feature A, length and weight conversion (`converter/length_weight.py`). Also created the repository and the ruleset.
- **Member B (Myriad Padri):** Feature B, temperature and volume conversion. Each member reviewed the other's work.

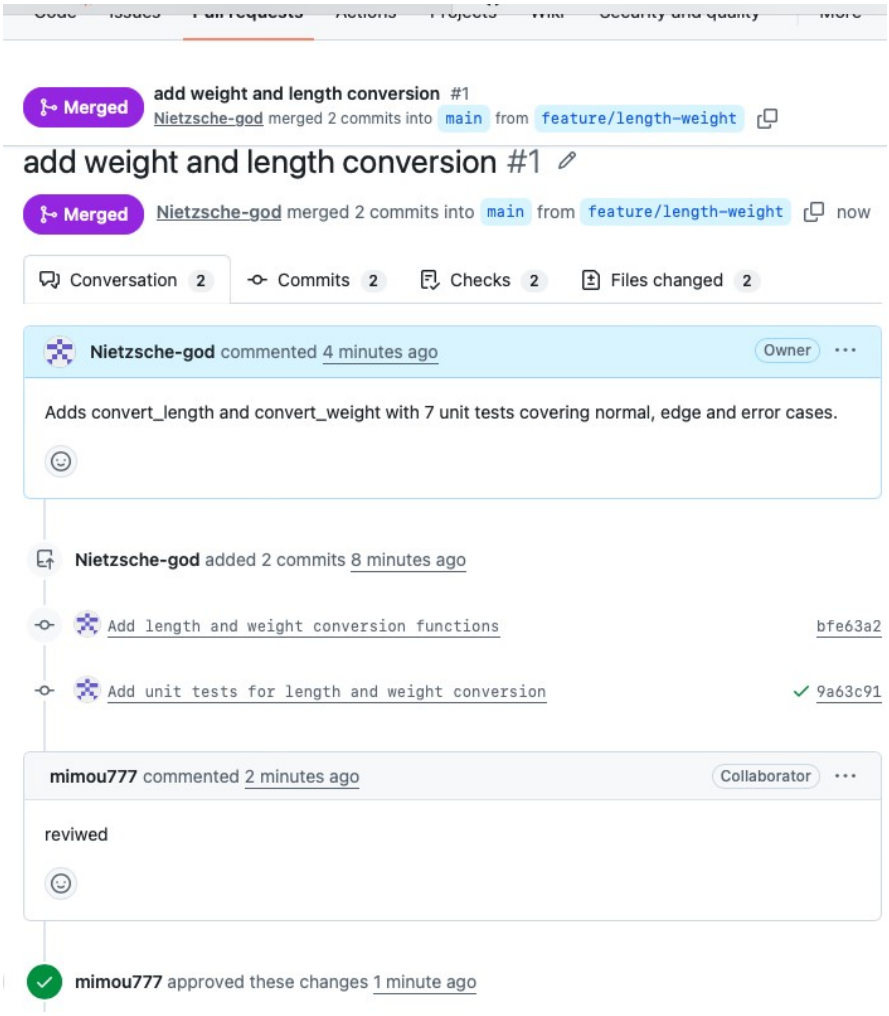
2. Branches, Commits & Contributions

Member A worked on `feature/length-weight`, started from the latest `main`. Meaningful commits: "Add length and weight conversion functions" (`bfe63a2`) and "Add unit tests for length and weight conversion" (`9a63c91`). Member A also created `status/a-update` for the README conflict exercise and `demo/blocked-pr` to demonstrate the required checks.

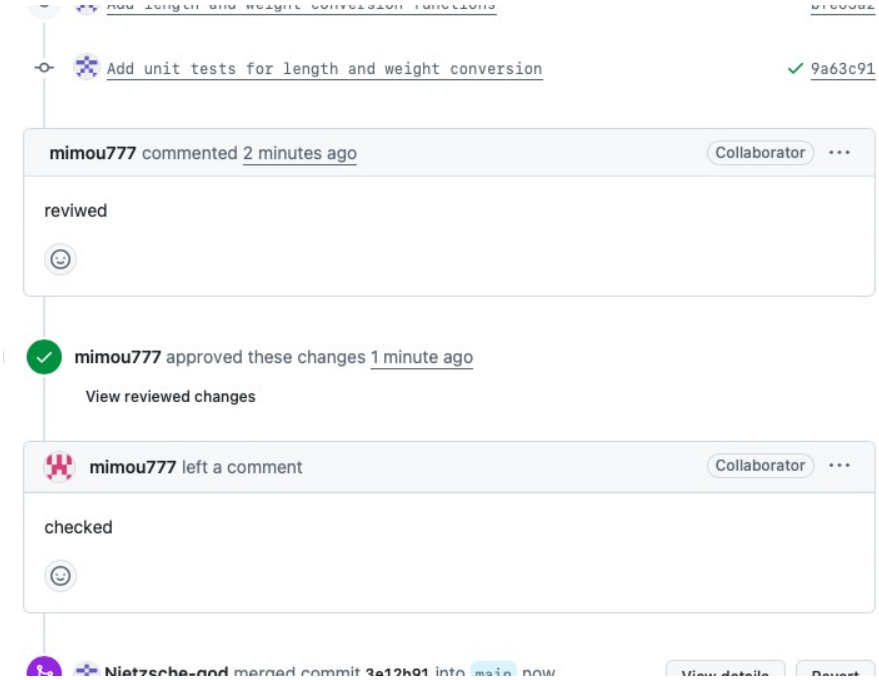
Member B worked on her own feature branch for temperature and volume conversion, with a commit for the feature code, a commit for its tests, a fix commit after the failed CI run, and an extra commit adding the Kelvin to Fahrenheit test requested in review. She also created `status/b-update` for the README conflict exercise.

3. Pull Requests & Code Review

Member A's PR (#1, "add weight and length conversion") was reviewed by Member B (`mimou777`), who left a comment and approved it. Member A then merged it.

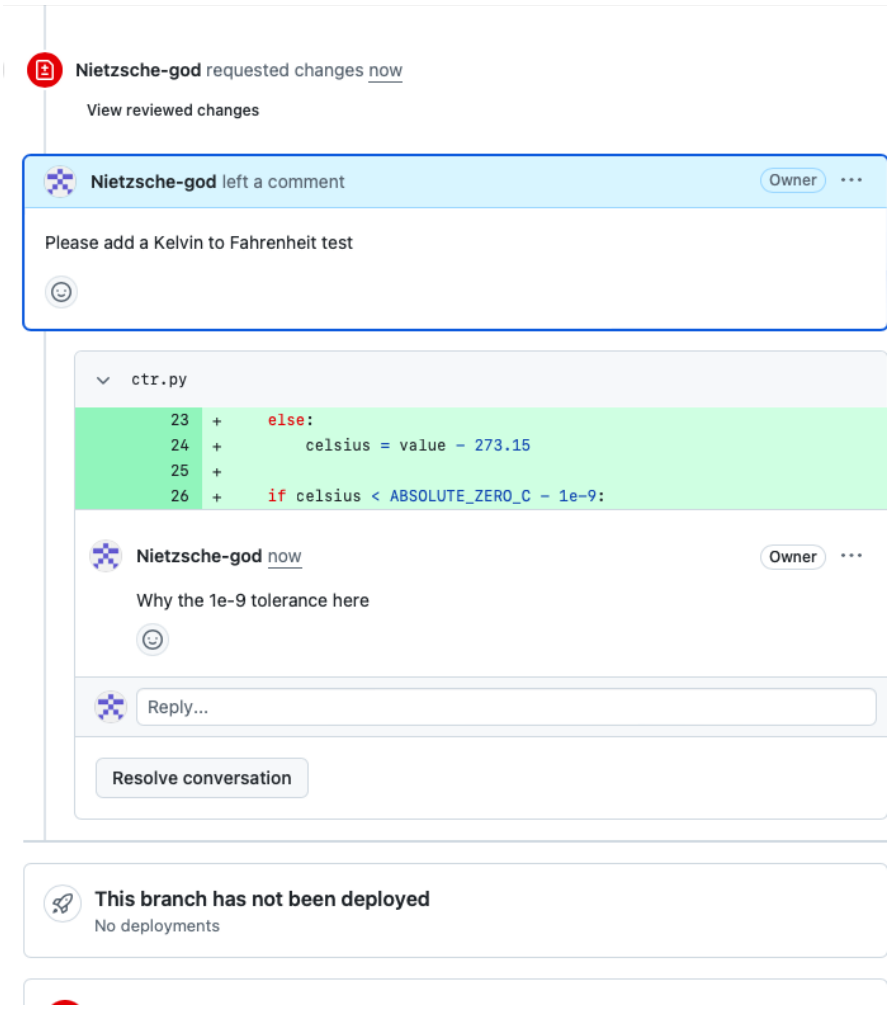


PR #1: review comment and approval by Member B



PR #1: approval, then merged into main

Member B's PR was reviewed by Member A. Member A left a line comment on the absolute-zero check ("Why the 1e-9 tolerance here") and submitted **Request changes**, asking for a Kelvin to Fahrenheit test. The PR was blocked from merging while the change was requested.



Member A's Request changes review and line comment on Member B's PR

Member B added the requested test on the **same branch**, committed and pushed it, and the existing PR updated automatically. Member A then re-reviewed and approved it before merging.

4. Merge Conflict

We added the line `Project status: Development in progress.` to `README.md`. Both members created a branch from the same version of `main`: `status/a-update` (A) and `status/b-update` (B), and each changed that exact line differently. Member A's PR was approved by Member B and merged first.

On `status/b-update`, Member B ran `git fetch origin` and `git merge origin/main`, which produced a conflict in `README.md`.

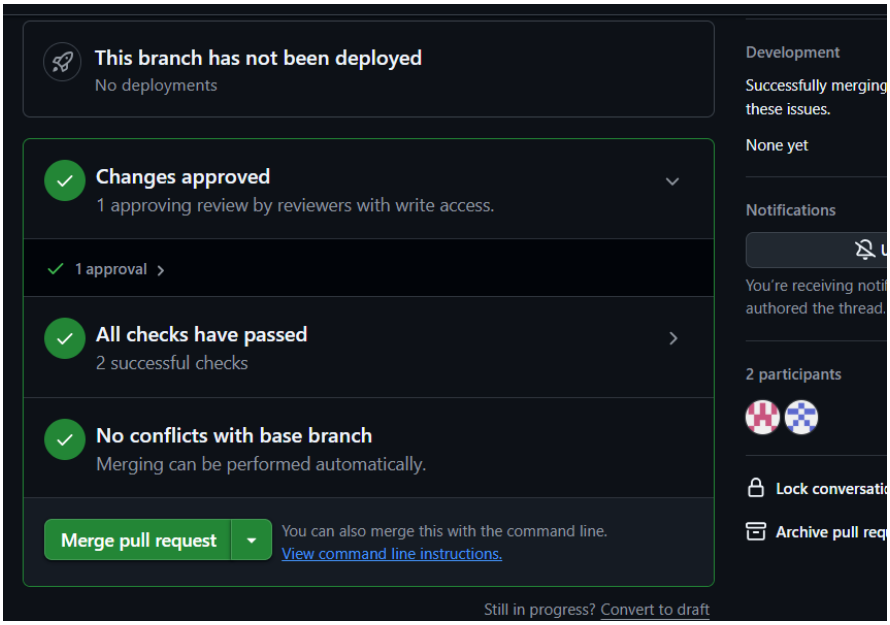
```
hp@DESKTOP-V3KCJCI UCRT64 ~/unit-converter (status/b-update)
$    git checkout status/b-update
   git fetch origin
   git merge origin/main
Already on 'status/b-update'
Your branch is up to date with 'origin/status/b-update'.
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 7 (delta 4), reused 3 (delta 2), pack-reused 0 (from 0)
Unpacking objects: 100% (7/7), 2.16 KiB | 64.00 KiB/s, done.
From https://github.com/Nietzsche-god/unit-converter
 * [new branch]      feature/length-weight -> origin/feature/length-weight
   d6a0ea4..0a49c18  main                  -> origin/main
 * [new branch]      mimou777-patch-1      -> origin/mimou777-patch-1
 * [new branch]      status/a-update       -> origin/status/a-update
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

Conflict on Member B's branch after `git merge origin/main`

In this case **Current (HEAD)** was Member B's line ("Temperature and volume converters complete") and **Incoming (origin/main)** was Member A's line ("Length and weight converters complete"). Both were true, so we combined them into one final line:

```
Project status: Length, weight, temperature and volume converters complete.
```

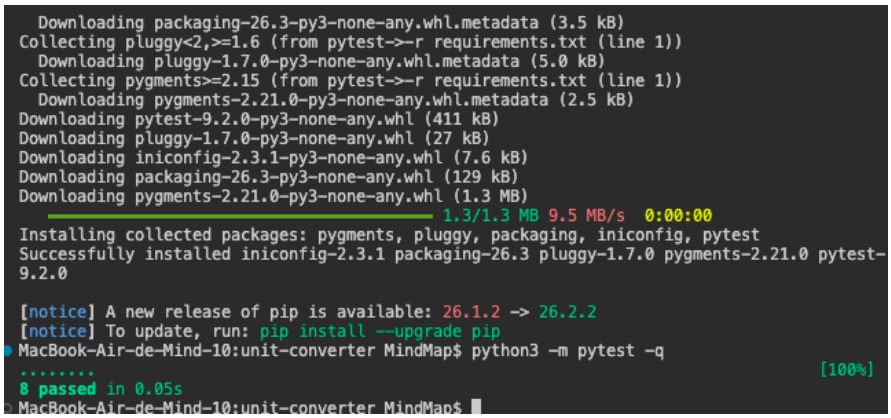
Member B staged `README.md`, committed the resolution, and pushed. Her PR then showed no conflicts, one approval and passing checks, and Member A reviewed and merged it (PR #4).



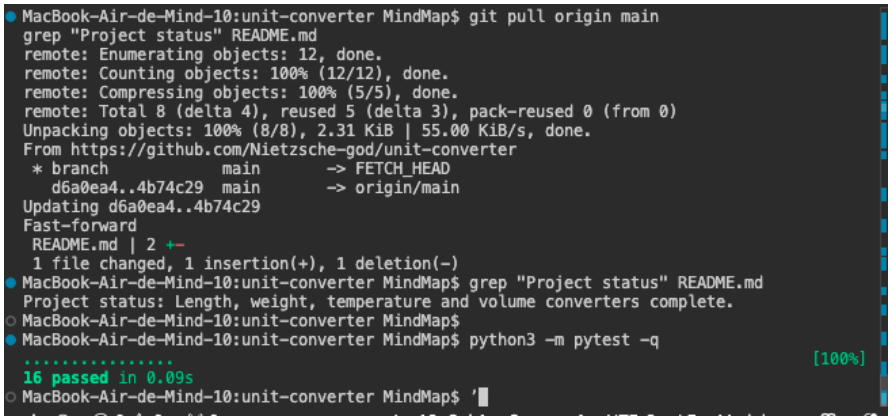
Member B's PR after resolution: approved, checks passed, no conflicts

5. Unit Tests

The test suite is in `tests/` and uses pytest. Locally, Member A's suite passed (8 tests including the smoke test), and after all features and the review-requested test were merged, the full suite on `main` passed **16 tests**.



Local pytest result for Member A's branch



Full suite on main after the merge conflict resolution: 16 passed

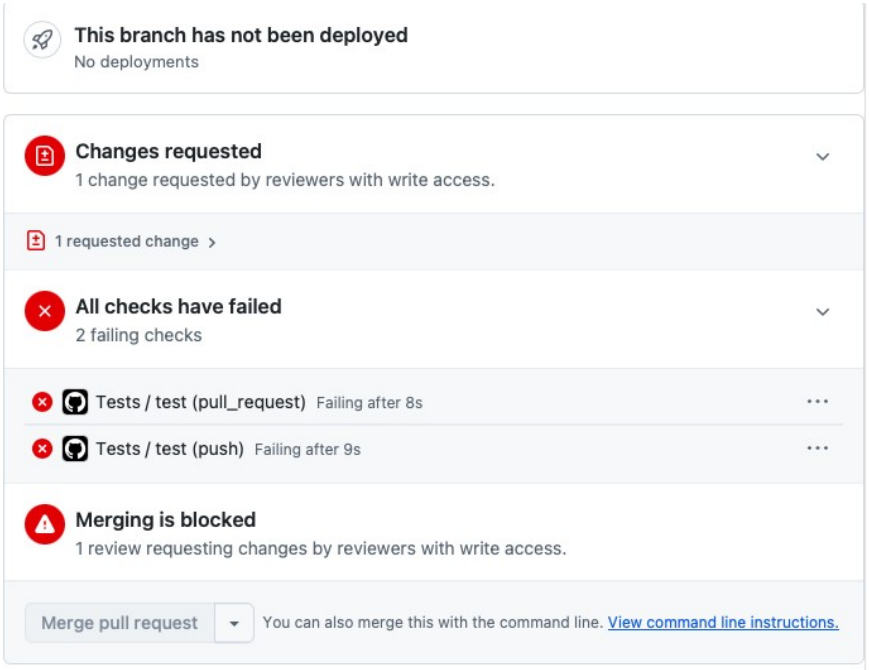
Examples from the suite:

- **Normal behavior:** `convert_length(2, "km", "m") == 2000`; `convert_temperature(100, "C", "F")` is 212; `convert_volume(1.5, "l", "ml") == 1500`.
- **Edge/boundary case:** zero length (`convert_length(0, "m", "foot") == 0`), converting a unit to itself, and absolute zero (`-273.15 C` equals `0 K`).
- **Invalid input/error handling:** unknown units and negative weights or volumes raise `ValueError`; a temperature below absolute zero raises `ValueError`.

6. Continuous Integration

The workflow `.github/workflows/tests.yml` (named "Tests") runs on every **push** and **pull request**. It has one job, `test`, on an `ubuntu-latest` runner with these steps: check out the repository (`actions/checkout`), prepare Python 3.12 (`actions/setup-python`), install dependencies (`python -m pip install -r requirements.txt`) and run `python -m pytest -q`.

Failed run: Member B's first PR failed CI (both the `pull_request` and `push` runs). The cause was a deliberately wrong expected value in one temperature test, so pytest reported a failed assertion in the Actions log. Member B fixed the expected value, committed and pushed to the same branch, and the checks passed on the next run.

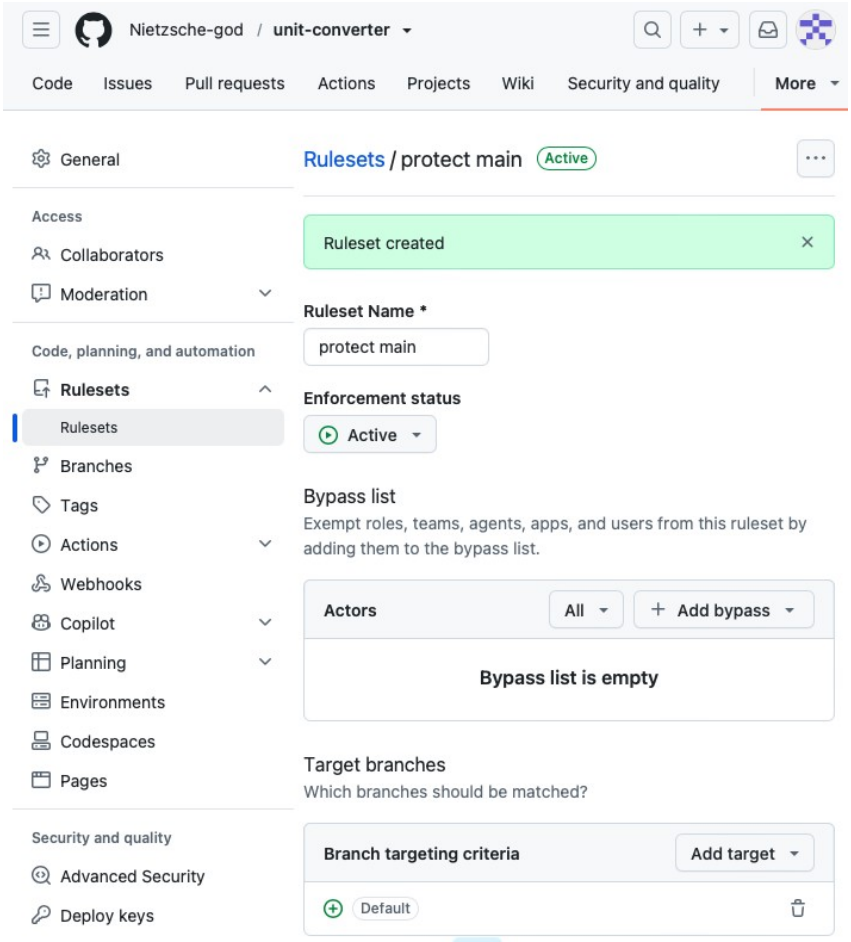


Failed CI on Member B's PR: two failing checks and merging blocked

7. Ruleset & Protected main

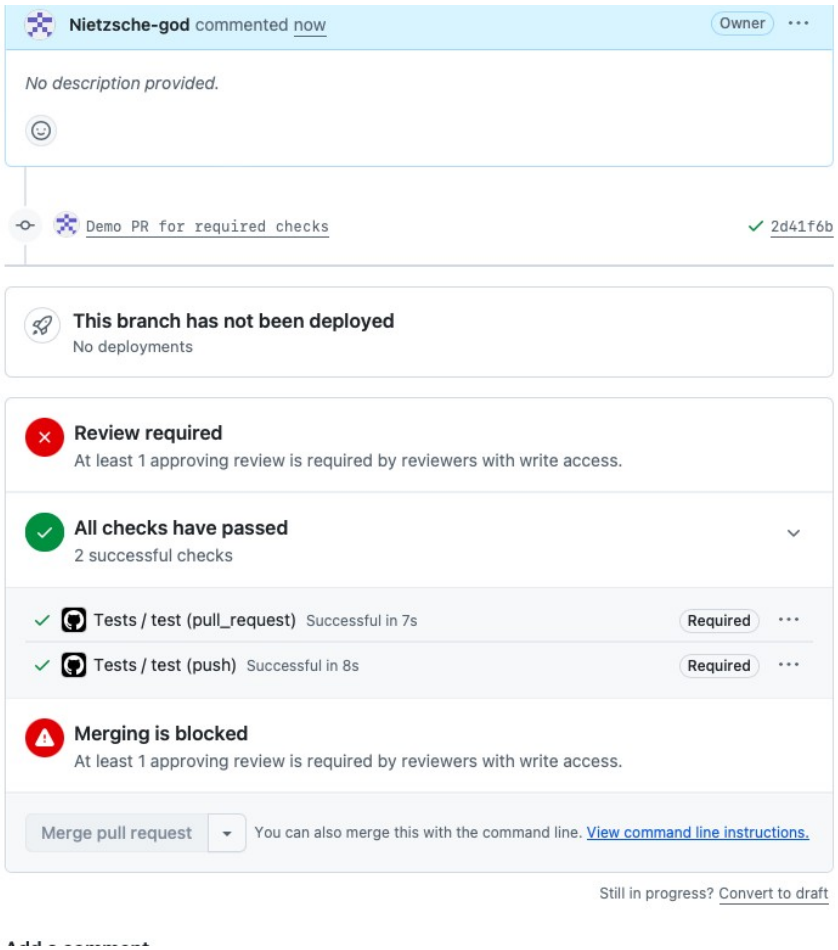
The active ruleset `protect main` targets the default branch (`main`) with an **empty bypass list**. It enforces:

- Require a pull request before merging, with at least **1 approval**.
- Restrict deletions and block force pushes.
- Require the status check `Tests / test` to pass (added after the workflow worked).



Active ruleset with empty bypass list

To demonstrate that both conditions are required, Member A opened a demo PR from `demo/blocked-pr`. Even with both required checks (`pull_request` and `push`) green, the PR showed **Review required** and **Merging is blocked** until another member approved.



Demo PR: checks passed but merging blocked until approval

```
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'demo/blocked-pr' on GitHub by visiting:
remote:      https://github.com/Nietzsche-god/unit-converter/pull/new/demo/blocked-pr
remote:
To https://github.com/Nietzsche-god/unit-converter.git
 * [new branch]      demo/blocked-pr -> demo/blocked-pr
branch 'demo/blocked-pr' set up to track 'origin/demo/blocked-pr'.
MacBook-Air-de-Mind-10:unit-converter MindMap$ git branch -f main origin/main
branch 'main' set up to track 'origin/main'.
MacBook-Air-de-Mind-10:unit-converter MindMap$ git status
On branch demo/blocked-pr
Your branch is up to date with 'origin/demo/blocked-pr'.

nothing to commit, working tree clean
MacBook-Air-de-Mind-10:unit-converter MindMap$ git log --oneline -3
2d41f6b (HEAD -> demo/blocked-pr, origin/demo/blocked-pr) Demo PR for required checks
4b74c29 (origin/main, main) Merge pull request #4 from Nietzsche-god/status/b-update
d16b20b Resolve README status conflict
MacBook-Air-de-Mind-10:unit-converter MindMap$
```

Branch history at the time of the demo PR

8. Continuous Deployment

GitHub Pages is configured under Settings, Pages, with the source set to **Deploy from a branch**: `main`, folder `/docs`. Every merge into `main` that changes `docs/` republishes the page

automatically. The page shows the project title, theme, team members, features and run instructions.

For the final change, we edited `docs/index.html` on the branch `docs/final-update`, opened a PR, got it approved with passing checks, and merged it. After about a minute the live site at the Pages URL showed the updated page.

9. Contribution Summary

	Member A: Mohamed Amine Bouamri	Member B: Myriad Padri
Features	Length and weight conversion	Temperature and volume conversion
Branches	`feature/length-weight`, `status/a-update`, `demo/blocked-pr`, `docs/final-update`	Temperature/volume feature branch, `status/b-update`
PRs authored	Length/weight feature (#1), status update (A), demo PR, final docs PR	Temperature/volume feature, status update (B) with conflict resolution
PRs reviewed	Member B's feature PR (Request changes, then approval); Member B's conflict PR	Member A's feature PR (comment and approval); Member A's status PR; final docs PR
Tests	7 tests for length and weight	7 tests for temperature and volume, plus the Kelvin to Fahrenheit test

10. Fair Use of AI

Both members used an AI assistant (Claude) during the assignment. It was used to propose a project structure, generate the starter code, tests, GitHub Actions workflow and project page, and to explain Git/GitHub steps one at a time (branches, pull requests, ruleset settings, `git fetch` and `git merge origin/main`). The prompts asked for step-by-step instructions, command explanations and help reading error messages. Member B additionally used her own AI conversation to troubleshoot errors she hit on her side. We accepted the generated code after checking it, replaced placeholders with our real names and usernames, wrote our own review comments, and deliberately changed one test expectation to produce the required failed CI run. Everything was verified by running `pytest` locally, reading the GitHub Actions logs and checking each pull request on GitHub before merging. Each member remains responsible for understanding the code, tests, commands, workflow and reviews done under their account.

11. Sources & Reflection

Sources: Session 2 course slides (Collaboration & Automation); GitHub Docs (rulesets, pull requests, GitHub Actions, GitHub Pages); pytest documentation; Claude (AI assistant) for guidance as described above.

Reflection: The hardest part was coordinating the merge conflict, because both branches had to start from the same `main` and the first PR could only be merged after the second branch was pushed. We also had to be careful that the ruleset blocked direct pushes to `main`, so every change, including the final report, had to go through a reviewed PR. If a PR, merge, test or workflow behaved unexpectedly, we would first check which branch we were on with `git status`, then read the failing step in the GitHub Actions log, and then check that the ruleset's required review and check conditions were actually satisfied.